

Project Report – Online Pong

CS371 – Fall 2025

Donnie Tkachenko, Daniel Hasselwander, Jackson Russell

Background

The goal of this project was to implement an online multiplayer pong game while utilizing a client-server architecture. The main task at hand was figuring out a way to allow two players to compete against each other, utilizing a network connection. Unlike usual single-system games, we needed to figure out a way to manage the game's states across two clients and keep them in sync, in which we learned and utilized socket programming along with threading.

Design

Prior to starting our implementation of code, we were given a handful of helper functions already, such as the UI and core game, and left with figuring out a way to handle the logic behind the server and client connections. We decided that for our architecture it would be better to use a client-network architecture for the code, where we can use a server as a relay rather than using a client as the host. This was done because it would mean that one client would have more computational loads. Utilizing it this way, all the computational loads would be left on the server for data relaying, while users are left with physics and rendering. We also had to solve the issue of synchronization, where the clients could be out of sync and one might have hit the ball, meanwhile the other might've missed, and it would cause a lot of issues, so the design had a synchronized counter server and client side to make sure the games are running smoothly. We used a connection-based, TCP socket protocol. Connection-based is not typical for game servers as latency is usually the main concern. Pong's simplicity, however, performs better with a connection-based protocol as the overhead adds no noticeable latency, and the minimal packet loss allows for smoother movement from the opponent and ball.

Implementation

We utilized a handful of python libraries, the first provided one was pygame for the actual game, then we also used socket and threading. For the server side of things, the server will

listen on a specific port and accept incoming connections. Depending on the order that clients join, the clients are assigned Left or Right paddles. Whenever a client connects, they're also assigned a new thread, which is the `readMessage` function. This thread will listen for packets from the clients and relay the received packets directly to the opposing client. For this method, we required a handshake before starting the game. This handshake provided initial information to the client in order to start the game. The sent data was window size and paddle position (left or right). The server waited to send this data until two clients joined as the client would immediately start the game upon receiving the initialization information. This allows for player one's game to wait to start until there's an opponent to play against.

For the client side of things, there's already the provided pre-existing game code, which handles all of the collision, inputs, and rendering. Each client was given a thread called `receive_updates`, which is used to receive the packets that the server relays to them from the opposing client. This unpacks the packet and updates the `shared_state`. The game state also has a `sync` function, which is also in the packet. This `sync` is just a counter to make sure all game instances are running on the same "frame". If the `sync` values are out of line, let's say you're game is running behind; your game would update with the latest received packet from the server.

Challenges

The biggest challenge that arose from this project was keeping the ball running smoothly on both the clients' screens and avoiding any sort of teleporting or weird behavior. This was a major issue in the beginning because we hadn't implemented the `sync` packet yet, and the clients would be connected to the same server, yet both of them would be seeing completely different things, they often times wouldn't even see the other client moving their paddle, or if they did, it'd be extremely far behind from when it actually happened. This is when we utilized the `sync` variable, which solved this issue, and even if it does get out of `sync`, it will update the game state to what's happening on the server currently.

This also caused a problem in itself. If you have both clients' sending data of what's happening on their screen, you get a race condition, and how do you differentiate which one is the right state? This is where we implemented a `shared_state` dictionary, this allowed us to check for flags called `should_update` which decide if the main thread should use the network data received.

The provided client code seemed to come up with the issue of the objects on the screen smearing or leaving a ghost trail as the new frame didn't seem to fully update the screen.

The provided code attempted to update only the objects written to the screen; however, since the location had already changed, it just left the object drawn in its old position too. This was fixed by simply refreshing the entire screen for every new frame instead of just the objects. This adds a cost, but Pong is very simple, so it doesn't affect gameplay. Since the pygame code wasn't written by us, this bug was hard to locate before fixing.

Lessons Learned

There were a handful of things that we learned from this project. The first and most important one being the actual implementation and utilization of threads and sockets. We were able to learn how daemon threads work and interact with program execution. We also learned about different kinds of sockets, whether they are blocking or nonblocking, along with edge cases of what to do whenever a socket or connection is disconnected. Another fun one to learn was Python's Structs to handle our data packet. It was interesting to learn how using struct.pack and unpack gave C levels of speed compared to text based packets like JSON or XML.

Conclusion

The Pong game successfully implements a client-server architecture for an online pong game. The game smoothly runs with two people playing against each other. Every aspect expected of us to demonstrate a working proof of concept for an online pong game seems to have been met. Extra features or improvements would be needed to run this code in a more production environment, such as the ideas suggested for bonus points. The work to complete this project had consistent improvements which can be seen in the GitHub repository with only one real moment of getting stuck on an issue and halting production. A relatively long amount of time was wasted by trying to implement multiple clients. There wasn't much planning ahead to do so, and thus it was poorly implemented by multiple people and ultimately failed to work, so it was scrapped.

List of Changes/Commits

Add More Given Files:

In this commit there were 9 files that were changed. Our helper code was set up (helperCode.py), along with our two main files that we worked on which were: pongClient.py and pongServer.py. The other 6 files were just looked over with no binary file being shown for them in our commits.

Set up Simple TCP Connection:

In this commit, the two files: pongClient.py and pongServer.py were edited. The main purpose of this commit is to build a server that can run and stop from a terminal. Set up a loop to look for STREAM connections and print anything sent. Client sends “hello server” on a connection. Works with one client and server.

Added Many Client Connections:

In this commit, once again only two files were edited: the Client and Server files. This edit allows multiple clients to now connect and run their own thread. Shutting down the server now closes every thread and socket for a more graceful exit.

Successful Game Start:

In this commit, Client and Server are edited again. This edit makes sure the starting information from the server is successfully sent to the first two clients. Along with making sure that the game no longer “smears” due to the screen refresh issue mentioned in the “Challenges” section of this document.

Cleaned and Documented:

This commit simply just deals with the cleaning up of our comments made along the way along with documentation regarding what was all done and by who.

Implemented Server Sync, Left vs Right Handshake in Client, Client/Server Updating, Packet Structure for Client:

In this commit there were several additions to the Client and Server files. The main purpose of this is to add some synchronized multiplayer support using 24-byte packets and a sync value. With these changes the client now runs a thread to receive opponent updates,

applies catch up logic, and sends its own score state each frame. Then the server was updated to relay these packets between clients, and assign left and right paddles on connect and handle shutdowns more cleanly.

Fixed Paddle Assigning and Pausing for Game Start:

In this commit, there are some small edits to Client and Server files. These edits wait for the handshake from the server to confirm the two players needed to start, along with now properly assigning paddles by alternating assignments on each client connection. This was implemented with the idea that 3rd or more clients would simply share control with either the left or right paddle, determined by the order the clients connected.

Cleaned up Code:

This commit cleaned up some of the previous code that didn't work as intended; the previous code was an attempt at a part of the bonus. These cleaned up bits reverted the current code back to a better version of the previous code before attempting the bonus.

=====